Training Models to Generate Training Environments: Stabilizing Bi-Level Curriculum Optimization

Robin Hylands
Western University
rhyland2@uwo.ca

Abstract

In this report I study the bi-level curriculum optimization problem for reinforcement learning in parameterized MiniGrid environments, where a curriculum generator selects training tasks for a PPO agent, and is evaluated and optimized based on the PPO agent's performance on a fixed benchmark of difficult target tasks. I compare a learned curriculum generated by a context free bandit policy, a random curriculum, and a no curriculum baseline. I establish a training approach that stabilizes the bi-level optimization problem, establishing the groundwork for more complex environment generation. I find several approaches that stabilize curriculum generation, allowing it to outperform the baselines. This provides a valuable base for future experiments improving curriculum, environment and reward generation models.

1 Introduction

The development of environments is a key aspect of reinforcement learning. Developing environments, agents, and creating reward functions to guide the agent behaviour is a difficult and time consuming process. Simultaneously, AI infrastructure compute capacity continues to grow at an exponential rate, while the evidence that approaches that leverage increasing compute end up dominating continues to grow. This naturally leads to the question, how can one make environment generation leverage compute gains more effectively? I propose an answer: leveraging increased compute to automate the process of environment generation.

This project is an introductory foray into automating the process of environment development for reinforcement learning agents. I hypothesize that a simple bi-level optimization process can improve the performance of generative models that create and augments environments for reinforcement learning agents. Simple bi-level optimization can be defined as an inner loop where an agent trains on tasks sampled from a generator, and an outer loop where the generator is updated based on agent performance on a set of target tasks.

1.1 Why Bi-Level Optimization is Hard

Bi-level optimization has several issues that impair its effectiveness in practice. The generator acts on a non resettable, history dependent learner, so the teacher is operating in a partially observed sequential control problem. The generator's choice at time t changes the agent state, which changes the value for all future tasks, the effect of which is only observable after many noisy training updates. This makes the outer objective delayed and noisy, and prioritizes tasks that produce short term improvements over long run transfer.

A second problem that emerges from simple bi-level optimization is the mapping from generator choices to final agent performance is very non-smooth. Small changes in task order can produce very different agent trajectories. Task spaces may be discrete or combinatorial, and the whole system

can have threshold effects where there is no observable performance gain until the agent crosses a capability boundary. This contributes to high variance and sometimes misguided outer loop gradients.

Thirdly, curriculum generators often suffer from mode collapse. The generator is susceptible to reward hacking, where it favors tasks near the agent's current frontier, forcing it to sample and update from a narrow band of tasks that reduce generator diversity over time.

1.2 Related Work

Existing techniques often approach this curriculum learning with evolution algorithms and tuning task difficulty as a proxy for contribution to agent ability. These approaches address the issues that emerge in bi-level optimization problems. Evolutionary algorithms do not require that the outer objective be smooth or even differentiable. They also naturally support task diversity by sustaining a population of policies instead of a single one. A population can support many different hypotheses about what a good curriculum looks like. POET uses evolution to build an expanding curriculum of diverse tasks (4). Eureka is a successful example of using evolutionary search over reward code, instead of gradient updates to the generator, and trains robot agents to perform very difficult tasks (5). Alternatively, Dennis et al. (6) tune an environment generator to maximize regret between two models, producing a curriculum of naturally evolving difficulty. Graves et al. (7) optimize a proxy for learning progress as well, adapting the curriculum using a nonstationary bandit driven by short-horizon learning-progress signals, thereby selecting tasks that appear immediately informative for the agent.

1.3 My Approach

Instead of trying to avoid the problems caused by bi-level optimization, this project seeks to find how the problem can be stabilized and used later for complex LLM based curriculum generators. Downstream agent performance is not the primary goal, instead I seek to improve generator performance, so that these generators can be used with existing techniques like Eureka with enhanced function. As such I work with the simple bi-level optimization problem with no work around. I choose to perform gradient based outer optimization instead of using evolutionary algorithms to improve environment generation.

2 Methodology

I study a bi-level curriculum optimization problem in which an outer *curriculum generator* learns which training tasks to present to an inner PPO agent, so that the agent ultimately performs well on a fixed benchmark of three hard target tasks. The two levels are kept cleanly separated: the generator never receives gradient information from the agent, and the agent never sees the generator's logits. Communication flows only through the agent's observable behaviour (benchmark success rate) after each inner training episode.

2.1 Environment and Task Space

All environments are built on MiniGrid (1), a lightweight partially-observable grid-world library. A task is fully specified by six discrete parameters:

Parameter	Values
`grid_size`	$\{5, 6, 7, 8, 9\}$
`num_rooms`	$\{1, 2, 4\}$
`num_keys`	$\{0, 1, 2\}$
`num_doors`	$\{0, 1, 2\}$
`num_lava`	$\{0, 1, 3, 5\}$
`goal_type`	{`reach_goal`, `pickup_ball`}

The Cartesian product yields up to $5\times3\times3\times3\times4\times2 = 1{,}080$ template configurations, filtered to those for which the grid has sufficient free cells to place all objects plus the agent spawn (roughly 750 valid tasks after filtering). Layouts support single-room open arenas, two-room environments

separated by a wall with up to one (optionally locked) door, and four-room environments with a cross wall and openings in each segment.

Benchmark tasks: Agent performance is evaluated on three fixed target tasks of escalating difficulty:

1. **Task 0.** 7×7 single room, 3 lava tiles, no keys or doors - requires deliberate hazard avoidance; not solvable by random walk.
2. **Task 1.** 8×8 two-room layout, 1 locked door, 1 key, no lava - requires finding the key before traversing the door.
3. **Task 2.** 9×9 four-room layout, 2 doors, 3 lava tiles - the hardest task, combining multi-room navigation with hazard avoidance.

All three are `reach_goal` tasks; the agent must reach a green goal square. The benchmark success rate is the mean success rate across the three tasks, averaged over 20 independent episodes each (60 episodes total per evaluation call).

2.2 Inner Loop: PPO Agent

The agent is a shared-trunk actor-critic network trained with Proximal Policy Optimization (PPO) (2). It receives MiniGrid's $7\times7\times3$ egocentric partial observation and the agent's cardinal direction (0–3) as input.

Architecture: The image is processed by a three-layer CNN:

$$\text{Conv}(3\to16,2)\to\text{ReLU}\to\text{Conv}(16\to32,2)\to\text{ReLU}\to$$
$$\text{Conv}(32\to64,2)\to\text{ReLU}\to\text{Flatten}\ \Rightarrow\ \mathbf{h}_{\text{img}}\in\mathbb{R}^{1024}$$

The direction is projected through a learned embedding $\mathbf{h}_{\text{dir}}\in\mathbb{R}^8$. The concatenated representation $[\mathbf{h}_{\text{img}};\,\mathbf{h}_{\text{dir}}]$ passes through a shared trunk (two fully-connected layers, 256 and 128 units, ReLU activations), producing features from which an actor head (7 action logits) and a critic head (scalar value) branch independently. All weights are orthogonally initialized; the actor output uses a small gain (0.01) to start near a uniform policy.

Training: At each inner iteration the agent collects $N_{\text{steps}}=4{,}096$ environment transitions across $N_{\text{envs}}=8$ parallel environments (512 steps per environment), computing advantages via Generalized Advantage Estimation (GAE, $\lambda=0.95$, $\gamma=0.99$) (3). Four passes of minibatch SGD (size 256) are then performed with a clipped surrogate objective ($\varepsilon=0.2$), a value loss coefficient of 0.5, and an entropy bonus coefficient of 0.01. Gradients are clipped to norm 0.5. The Adam optimizer is used with learning rate 2.5×10^{-4}.

Two agent-evaluation protocols: I tested two approaches to tying the inner-loop PPO agent to the outer-loop curriculum generator.

1. **Progressive lifelong agent protocol.** A single PPO agent instance is shared across the entire outer loop, and its weights are not reset between outer iterations. This design rewards the curriculum generator for helping the agent progress from early to late training. This allows long term transfer to be rewarded, but also introduces a moving target problem, as the agent changes over time, while the generator remains a context free bandit policy (see section 2.3).
2. **Steady-state fresh-agent protocol.** Each sampled curriculum task is evaluated using freshly initialized agents that all start from the same saved initial PPO weights. For each sampled task I instantiate 3 new agents, and train each one for the same fixed inner-loop budget, and average their benchmark evaluations. This converts the outer-loop objective into a steadier "help a weak agent at a fixed stage of development" problem rather than a moving-target "track one agent from early to late learning" problem. This prevents the generator from being rewarded for tasks that help long term success, but provides a simpler, fixed target that isolates the curriculum generator's ability to improve agents at a particular development stage and reduces variance from agent-history effects.

2.3 Outer Loop: Curriculum Generator

The curriculum generator is a *factored categorical bandit policy*: each of the six template parameter axes has its own independent categorical distribution represented as a learnable logit vector, initialized to zero (uniform). The joint sampling distribution is the product of these six marginals, so the generator can be represented as $\pi_\phi(\tau) = \prod_{k=1}^{6} \pi_{\phi_k}(\tau_k)$. At each outer iteration the generator samples a batch of $B = 32$ tasks $\{\tau^{(i)}, \log \pi_\phi(\tau^{(i)})\}_{i=1}^{B}$.

Subtask reward shaping: Alongside each task, a *subtask selector* — a separate learned categorical distribution over four shaping strategies — selects a shaped-reward mode to apply during the agent's inner rollout. The four modes are:

- **none** — no auxiliary reward (pass-through).
- **pickup_key** — one-time bonus of 0.2 the first time the agent picks up a key.
- **open_door** — one-time bonus of 0.2 the first time a door is opened.
- **explore** — per-step bonus of $0.2/n_{\text{interior}}$ for each previously unvisited cell, so the total potential shaping bonus across a full grid is 0.2.

The subtask shaping wrapper is applied only during inner training rollouts; the benchmark evaluation always uses the unshaped environment reward.

2.4 Generator Reward Signal

The generator's reward for presenting task $\tau^{(i)}$ is defined as

$$r^{(i)} = \Delta s^{(i)} - \lambda_d \cdot \mathbf{1}[\text{too easy or too hard}], \tag{1}$$

where $\Delta s^{(i)} = s^{(i)}_{\text{post}} - s_{\text{prev}}$ is the change in mean benchmark success rate observed immediately after the agent trains on task $\tau^{(i)}$, $\lambda_d = 0.3$ is a difficulty-penalty weight, and the indicator fires when the agent's within-task success rate on $\tau^{(i)}$ falls below 5% (too hard) or exceeds 90% (too easy). This "Goldilocks" penalty is designed to encourage the generator to select tasks at the productive frontier of the agent's current ability.

Raw rewards are smoothed with an exponential moving average (EMA) before use in the REINFORCE update:

$$\hat{r}^{(i)} \leftarrow (1-\alpha)\,\hat{r}_{\text{prev}} + \alpha\, r^{(i)}, \quad \alpha = 0.25.$$

This reduces the variance introduced by the noisy inner-loop evaluation while still tracking recent performance trends.

2.5 Policy Updates

Both the generator π_ϕ and the subtask selector are updated by REINFORCE with an exponential moving average baseline for variance reduction. Given a batch of B samples $\{(\log \pi_\phi(\tau^{(i)}),\, \hat{r}^{(i)})\}$, the generator loss is

$$\mathcal{L}_{\text{gen}} = -\frac{1}{B}\sum_{i=1}^{B} \log \pi_\phi(\tau^{(i)})\,(\hat{r}^{(i)} - b) - \beta_H\, H(\pi_\phi),$$

where b is the EMA reward baseline ($\alpha_b = 0.05$), $H(\pi_\phi)$ is the distribution entropy of the generator (computed as the sum of per-axis entropies), and $\beta_H = 10^{-3}$ is an entropy regularization coefficient that discourages mode collapse. An identical REINFORCE loss with the same EMA baseline is used for the subtask selector. Both are optimized with Adam (generator LR 5×10^{-3}; subtask selector LR 5×10^{-3}); gradients are clipped to norm 1.0.

Negative-task suppression: The generator maintains a per-task exponential moving average score ($\alpha_{\text{task}} = 0.15$). At sampling time, any task whose running score falls below -0.05 is rejected with probability 0.80, reducing the chance of repeatedly sampling tasks that have consistently harmed benchmark performance.

2.6 Experimental Sequence and Variance Reduction

I ran a sequence of variants whose primary purpose was to reduce the variance and non-stationarity of the signal sent to the outer-loop curriculum generator.

Stage 1: Baseline bi-level curriculum optimization. The initial experiment used the progressive lifelong-agent protocol together with the full generator reward, including subtask shaping and the difficulty penalty.

Stage 2: Stabilized lifelong-agent curriculum. To reduce noise while preserving the original formulation, I increased the number of evaluation episodes, increased the generator batch size, lowered the generator and selector learning rates, added entropy regularization, smoothed the reward with an EMA, suppressed repeatedly harmful tasks, and restricted the generator to `reach_goal` tasks, removing the pickup_ball subtask.

Stage 3: Reward ablations. After observing that curriculum collapse persisted even under the stabilized setup, I ablated the two main reward-design components: sub task shaping and the difficulty penalty. These ablations were motivated by variance reduction and preventing generator mode collapse: both components add additional terms to the generator reward that can amplify noise or bias the generator toward short-term gains, and produce reward hacking behaviour.

Stage 4: Hyperparameter sweeps after ablation. Once the outer-loop reward was simplified, I varied the generator learning rate, batch size, and outer-loop length to study the trade-off between stability and adaptivity. I reduced the number of outer iterations to save time under my time constraints. These runs were used to determine whether the generator was under-reacting or over-reacting to its reward signal after the shaping terms were removed.

Stage 5: Steady-state fresh-agent protocol. The final experimental change targeted the largest remaining source of non-stationarity: the agent itself. Instead of evaluating the generator on one persistent agent, each sampled curriculum was evaluated on multiple freshly initialized agents starting from the same saved initial weights.

Table 1: Key protocol and hyperparameter settings for each stage of the experimental sequence.

Stage	Protocol	LR	Batch	Outer iters	Shaping	Difficulty penalty	Stabilization
1	Lifelong	3×10^{-2}	8	200	On	On	Off
2	Lifelong	1×10^{-2}	16	200	On	On	On
3	Lifelong	1×10^{-2}	16	200	Off	Off	On
4.1	Lifelong	5×10^{-3}	32	100	Off	Off	On
4.2	Lifelong	8×10^{-3}	32	100	Off	Off	On
5	Reinit	8×10^{-3}	32	100	Off	Off	On + avg

All conditions use the same random seed for reproducibility. Results are reported on the three fixed benchmark tasks described above, with mean late stage success rate as the primary metric of performance.

2.7 Experimental Baselines

Three training conditions are compared, each with the same PPO agent architecture hyperparameters and compute budget to establish baselines to compare my approach against:

Curriculum: the generator and subtask selector are trained jointly as described above.

Random: tasks are sampled uniformly at random from the valid template space. This baseline controls for the benefit of task diversity alone.

None: the agent trains exclusively on tasks drawn uniformly from the three benchmark tasks themselves, with no generated curriculum or subtask shaping.

Table 2: Key hyperparameters.

Group	Parameter	Value
Environment	Parallel envs per rollout (N_{envs})	8
	Inner steps per task (N_{steps})	4,096
	Eval episodes per task	20
PPO	Discount γ	0.99
	GAE λ	0.95
	Clip ε	0.2
	Entropy coef	0.01
	Value loss coef	0.5
	PPO epochs	4
	Minibatch size	256
	Learning rate	2.5×10^{-4}
	Grad clip norm	0.5
Outer loop	See Table 1	

2.8 Remaining Hyperparameter Summary

3 Results

I will now present the results over the progression of design choices. I found that focusing on just environment generation and removing reward shaping yielded the best results. Furthermore, I found that the gamut of stabilization techniques I applied noticeably improved the curriculum generator's performance. I also found that generator performance with a non moving curriculum target performed comparably to the moving target.

Table 3: Final benchmark success rates across curriculum experiment variants. Curriculum refers to the learned curriculum, Random refers to the random curriculum, and None refers to no curriculum.

Stage	Outer Loop Iterations	LR	Batch	Shaping	Stabilization	Curriculum	Random	None
1	200	3×10^{-2}	8	On	Off	0.067	0.300	0.233
2	200	1×10^{-2}	16	On	On	0.117	0.300	0.233
3	200	1×10^{-2}	16	Off	On	0.350	0.283	0.217
4.1	100	5×10^{-3}	32	Off	On	0.183	0.417	0.217
4.2	100	8×10^{-3}	32	Off	On	0.383	0.250	0.367

Table 4: Late-stage benchmark success averages across curriculum experiment variants. Curriculum refers to the learned curriculum, Random refers to the random curriculum, and None refers to no curriculum. Success rates are averaged over the final quarter of training.

Stage	Outer Loop Iterations	LR	Batch	Shaping	Stabilization	Curriculum	Random	None
1	200	3×10^{-2}	8	On	Off	0.032	0.321	0.292
2	200	1×10^{-2}	16	On	On	0.055	0.321	0.292
3	200	1×10^{-2}	16	Off	On	0.245	0.334	0.299
4.1	100	5×10^{-3}	32	Off	On	0.271	0.266	0.301
4.2	100	8×10^{-3}	32	Off	On	0.235	0.221	0.297

3.1 Collapse Under Reward Shaping

I found that the curriculum's performance collapses with simple reward shaping. The diversity of generated curriculum also experienced mode collapse, indicating that the model is reward hacking the shaped rewards to the detriment of overall agent performance. I found that employing various

stabilization techniques greatly improved the curriculum generator's performance and diversity, but it still lagged behind the random curriculum and no curriculum baselines.

Ultimately, simply removing the reward shaping from the curriculum design successfully made the curriculum generator competitive with the baselines.

3.2 Further Improvements

Furthermore, I found that increasing the batch size of the inner loop, and tuning the learning rate, allowed the learned curriculum to make further gains, surpassing the random curriculum. I found that when training for 200 outer loop iterations, the random curriculum surpasses the no curriculum baseline, but because of time constraints, I wasn't able to train the newer experiments for 200 iterations, and thus while the learned curriculum outperforms the random curriculum in these runs, it does not catch up to the no curriculum agents by the end of 100 outer loops, although I expect under a full training run of 200 episodes, the updated learned curriculum agents will outperform the no curriculum baseline as does the random curriculum in experimental stage 3. This prediction is tentative however, as it assumes transitivity of relative gains across stages. See Figure 1 for the success over the 100 outer optimization iterations of each technique in stage 4.2.

I attribute the improved performance in Stage 4 to the increase in the batch size of the outer optimization loop, which further stabilizes the training.

Figure 1: Success rate over 100 iterations of outer optimization during experiment stage 4.2.The learned curriculum (blue) rises above the random baseline (orange) by the end of training."

3.3 Reinitializing Agent Models

I test curriculum learning with a more stationary target of freshly initialized PPO agents, and find that the differences are within noise given a single seed, but the ordering is consistent with my hypothesis.

Table 5: Performance of the final reinitialized-agent experiment (seed 44). Gain is measured relative to the average fresh-agent baseline success recorded during the run, which is the most appropriate improvement metric for the steady-state evaluation protocol.

Method	First-10 Avg	Last-10 Avg	Gain Over Baseline
Curriculum	0.247	0.259	+0.012
Random	0.241	0.251	+0.010
None	0.251	0.259	+0.008

Notably, I observe a trend consistent with the curriculum generator improving, though the effect is within noise, with success improving modestly from an early average 0.247 to a late average of 0.259.

4 Discussion

4.1 Comparison to Existing Approaches

Empirically, the Stage 1 results reproduce the failure mode that motivates evolutionary and proxy approaches: a naive REINFORCE outer loop with shaped rewards collapses to 0.067 benchmark success -well below both the random (0.300) and no-curriculum (0.233) baselines. When reward shaping is removed and stabilization techniques are applied (Stage 4), the gradient-based outer loop reaches a 0.271 average success, greater than the random baseline, with the potential to surpass the no-curriculum baseline with more training. This suggests that a gradient based curriculum generation optimization is a viable alternative to evolutionary and proxy approaches when the reward signal is kept simple, and sufficient stabilization techniques are implemented.

4.2 Next Steps

I note several limitations that suggest immediate next steps for the development of this project. The first limitation is the small number of iterations of the outer loop. I chose a smaller outer loop to allow for more iterations in the hyperparameter search, and now that I have settled on a functional experimental groundwork, future work should increase the number of outer loops.

The second major limitation is the simplicity of the curriculum generator and the template system for permuting the environment. This provides a very limited space for optimization that may create an artificial ceiling on optimization capabilities. Future work should experiment with more complex, and especially context dependent curriculum generation models. This naturally progresses towards the future goals of this project, eventually culminating in the use of an LLM for the environment generator. This will allow environment generation and augmentation to be open ended and code based, rather than template based.

A parallel between this stage of the project and the future is the need to control the generation of invalid environments. In this project I simply filter logically invalid environments, but a LLM environment generator can incorporate valid/invalid environment generation into its training, as has been explored in code-generation systems that learn directly from compiler or test-execution feedback (8). Incorporating valid and invalid environment generation thus presents an attractive direction for future work.

The high level goal of this project is not to improve agent models directly, but to instead to increase generative models' ability to generate environments. I envisage using these techniques to improve the generative models used in approaches like Eureka, so that a machine learning practitioner can plug a tuned model into the Eureka (or similar) algorithm and achieve performance increases over the base generative models used in these techniques.

5 Conclusion

I establish a baseline for enabling bi-level optimization of an agent-task-generator system. I tuned the training environment to facilitate a learned curriculum that outperforms the random baseline and reaches at least parity with the no curriculum baseline, which I further expect it to outperform given

more training. I do this to establish a groundwork for more complex applications, namely, the use of LLMs to generate environments automatically for downstream applications.

References

[1] Maxime Chevalier-Boisvert, Lucas Willems, Suman Pal, and Soroush Mehri. MiniGrid & MiniWorld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks. arXiv preprint arXiv:2306.13831, 2023.

[2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal Policy Optimization Algorithms. arXiv preprint arXiv:1707.06347, 2017.

[3] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[4] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. Paired Open-Ended Trailblazer (POET): Endlessly Generating Increasingly Complex and Diverse Learning Environments and Their Solutions. arXiv preprint arXiv:1901.01753, 2019.

[5] Yecheng Jason Ma, William Liang, Guanzhi Wang, Shuran Song, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Eureka: Human-Level Reward Design via Coding Large Language Models. arXiv preprint arXiv:2310.12931, 2023.

[6] Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre M. Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent Complexity and Zero-Shot Transfer via Unsupervised Environment Design. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] Alex Graves, Marc G. Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated Curriculum Learning for Neural Networks. In *International Conference on Machine Learning (ICML)*, pages 1311–1320, 2017.

[8] Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C. H. Hoi. CodeRL: Mastering Code Generation Through Pretrained Models and Deep Reinforcement Learning. arXiv preprint arXiv:2207.01780, 2022.